

Table of Contents

RayguiObjC — User Manual....1

1. What this project is....1
2. Immediate-mode GUIs, explained....2
3. Setup: building the library and the demo....3
4. Your first app....6
5. Coordinates, rectangles, and colors....7
6. How a control reports interaction (RGGResult)....8
7. The bound-state pattern....10
8. Strings, arrays, and memory rules....10
9. Controls reference....11
10. Editable text: the editMode pattern....16
11. Dialogs: message boxes and text input....17
12. Scrollable areas....18
13. Styling: themes, colors, fonts, icons, tooltips....19
14. Complete example: a settings window....22
15. Platform notes....24
16. Troubleshooting....24
17. Further reading....26

RayguiObjC — User Manual

How to build a graphical user interface for an Objective-C application with **RayguiObjC**, from scratch. This manual assumes **no prior knowledge of raylib or raygui** — both are introduced where they matter.

---

1. What this project is

RayguiObjC sits between your Objective-C code and two C libraries:

Layer	What it is	What you do with it
raylib 6.0	A C library that opens a window, draws shapes and text, reads the mouse and keyboard, and runs the per-frame loop.	Call it **directly** from Objective-C (`InitWindow`, `BeginDrawing`, `DrawText`, …). It needs no wrapper: Objective-C is a strict superset of C.
raygui 5.0	A C library of ready-made widgets (buttons, sliders, text boxes, …) built on top of raylib.	You normally **do not call it directly**. RayguiObjC wraps it.
RayguiObjC (this project)	A thin Objective-C wrapper around raygui's ~35 control functions.	This is the GUI API you program against: `[RGGui button:… text:…]`, `[RGGui slider:… value:…]`, …

Why wrap raygui at all? Its C API forces you to juggle `char *` strings, fixed-size stack buffers for text editing, and raw integer result codes. RayguiObjC gives you `NSString`, `NSArray`, `NSMutableString`, and `BOOL`, while raylib's structs (`Rectangle`, `Color`, `Vector2`, `Font`) pass straight through the boundary unwrapped — that is the entire point of this design.

Facts you should know before writing any code:

- **It is not a Cocoa/AppKit app.** Your program has a plain `main()` function. raylib creates and manages the window itself. There is no `NSApplication`, no storyboard, no Auto Layout.
- **Foundation only.** The wrapper uses `NSObject`/`NSString` and nothing from AppKit, so the *same source file* compiles on macOS (Cocoa Foundation) and on Linux (GNUstep Foundation) with no `#ifdefs`.
- **Also builds for the web.** The same source compiles to a WebAssembly webapp with emscripten (`Makefile.wasm`, §3.6) — libobjc2 as the ObjC runtime plus a minimal Foundation shim instead of Cocoa/GNUstep.
- **No ARC.** Everything is built with `-fno-objc-arc` (manual retain/release) so the same code compiles under GNUstep. Compile your own `.m` files the same way (see §3.3).
- **Pinned versions.** raylib is pinned at tag `6.0`, raygui at tag `5.0` (plus one cherry-picked upstream bug fix). `VERSIONS.md` is the single source of truth. Never build against `master` of either upstream repo.

---

2. Immediate-mode GUIs, explained

Most GUI frameworks you may know (Cocoa, UIKit, Qt) are **retained mode**: you create a button object once, the framework stores it, redraws it when needed, and notifies you through delegates/callbacks when it is clicked.

raygui is **immediate mode**. There are no widget objects. Every frame, you *call* a function for each control; that call reads the current mouse/keyboard state, draws the control, and returns what happened. The control's state (is the checkbox ticked? what is the slider value?) lives in **your** variables, which you declare once, outside the frame loop.

```
// Retained mode (Cocoa) — NOT how raygui works:
//   UIButton *b = [[UIButton alloc] init...];  // object persists
//   [b addTarget:self action:@selector(clicked:)];  // callback later

// Immediate mode (raygui) — how RayguiObjC works:
BOOL myChecked = NO;                    // state lives in YOUR variable
while (!WindowShouldClose()) {
```

```
    BeginDrawing();
        [RGGui checkBox:RGRectMake(20, 20, 24, 24) text:@"Enabled"
checked:&myChecked];
        // myChecked is updated right here, right now, if the user clicked
    EndDrawing();
}
```

Why bother? Because the UI code reads top-to-bottom exactly like the screen looks. No outlets, no delegates, no layout pass, no stale widget state. The cost is that you must follow the **frame-loop contract**:

1. Call each control **exactly once per frame**.
2. Call controls **only between `BeginDrawing()` and `EndDrawing()`**. raygui reads raylib's input state and draws through raylib's renderer; both are invalid outside the frame.
3. Keep the control's state in your own variables, declared **outside** the loop. The wrapper holds no per-control state.
4. One frame looks like this:

```
InitWindow(...)                     // once, at startup
SetTargetFPS(60)                    // once, at startup

while (!WindowShouldClose()) {      // once per frame:
    @autoreleasepool {              // drains the per-call autoreleased NSStrings
        // 1. Update   — your game/app logic, IsKeyPressed(), physics, …
        // 2. BeginDrawing()
        // 3. Controls — one call per control, top to bottom
        // 4. EndDrawing()
    }
}

CloseWindow()                       // once, at shutdown
```

A minimal skeleton with this shape is the whole skeleton of every RayguiObjC app — §4 fills it in.

---

3. Setup: building the library and the demo

3.1 Prerequisites

macOS: Xcode Command Line Tools (`xcode-select --install`) for clang and make. Install `cmake` separately if you don't have it (e.g. `brew install cmake`).

Linux (Debian/Ubuntu):

```
sudo apt install clang cmake gnustep-devel \
    libx11-dev libxrandr-dev libxinerama-dev libxcursor-dev libxi-dev \
    libxext-dev libgl-dev libasound2-dev libpulse-dev
```

(The X11/GL/ALSA packages are what raylib links against; `gnustep-devel` provides Foundation and `gnustep-config`.)

3.2 Fetch the pinned dependencies

```
git clone <this-repo> raygui_objc_wrapper
cd raygui_objc_wrapper
sh scripts/init-submodules.sh
```

The script clones `vendor/raylib` at tag `6.0` and `vendor/raygui` at tag `5.0`, verifies both pins, and applies one cherry-picked upstream fix (the `GuiColorPicker` hue-bar fix) as a working-tree patch. Afterwards `vendor/raygui` shows as locally modified — that is intentional; do not commit or reset it. See `VERSIONS.md`.

3.3 Build raylib (once per machine)

raylib is built statically; the wrapper links against the archive:

```
cd vendor/raylib
cmake -B build -DCMAKE_BUILD_TYPE=Release -DBUILD_SHARED_LIBS=OFF \
      -DBUILD_EXAMPLES=OFF -DBUILD_GAMES=OFF
cmake --build build --config Release
cd ../..
```

This produces `vendor/raylib/build/raylib/libraylib.a`.

3.4 Build and run the demo

```
make          # builds examples/demo/demo
make run      # builds (if needed) and runs it
make test     # builds and runs the tests in tests/ (unit + hidden-window smoke)
make deps     # fetch/pin submodules and build raylib (the two setup steps)
make clean    # remove build artifacts
```

`Makefile` and `GNUmakefile` are intentionally identical; each dispatches on `uname -s`, so the same `make` works on macOS (clang + Cocoa) and Linux (GNUstep). The demo opens an 800×450 window with a label, a button that toggles a checkbox, a toggle, a slider bound to a float, a spinner, a combo box, a text box, and a status bar.

To see a different theme, pass a style file as the first argument — raygui ships several under `vendor/raygui/styles/`:

```
./examples/demo/demo vendor/raygui/styles/cyber/style_cyber.rgs
./examples/demo/demo vendor/raygui/styles/dark/style_dark.rgs
```

(Binary `.rgs` files embed their font and load standalone. See §13.)

3.5 Building your own app against the wrapper

The `make` route in §3.4 is the quick start. The commands below are for wiring the wrapper into your own build system (or an Xcode project).

Your app is one or more `.m` files plus the wrapper sources. The wrapper is five files: `src/RayguiImplementation.m` (the *only* file that defines `RAYGUI_IMPLEMENTATION` — never define it yourself), `src/RGGControls.m`, `src/RGGContainers.m`, `src/RGGAdvanced.m`, `src/RGGState.m`, and the public header `src/RayguiObjC.h`.

macOS:

```
clang -x objective-c -fno-objc-arc -std=c11 -Wall \
  src/*.m myapp/main.m \
  -Isrc -Ivendor/raygui/src -Ivendor/raylib/src \
  -x none vendor/raylib/build/raylib/libraylib.a \
  -framework Foundation -framework Cocoa -framework IOKit \
  -framework CoreFoundation -framework OpenGL \
  -o myapp
```

Linux (GNUstep):

```
CFLAGS="$(gnustep-config --objc-flags) -Isrc -Ivendor/raygui/src
-Ivendor/raylib/src"
clang -x objective-c -fno-objc-arc -std=c11 -Wall $CFLAGS \
  src/*.m myapp/main.m \
  -x none vendor/raylib/build/raylib/libraylib.a \
  -lX11 -lXrandr -lXinerama -lXcursor -lXi -lXext -lGL -lrt -lm -lpthread -ldl \
  $(gnustep-config --base-libs) \
  -o myapp
```

Three details that bite people:

- The `-x none` before `libraylib.a` matters. `-x objective-c` applies to **every** input file that follows it, so without `-x none` clang parses the archive as Objective-C source and fails with `!<arch>` errors. (The alternative is two steps: compile each `.m` to `.o` with `-x objective-c`, then link with no `-x` at all — that is what the Makefile does.)
- **Never** `#define RAYGUI_IMPLEMENTATION` in your own code. That macro generates the definitions of all raygui functions; defining it in two translation units produces duplicate-symbol linker errors.
- Compile your own `.m` files with `-fno-objc-arc` too (as above). The wrapper is manual retain/release; mixing ARC files into the same binary is possible but the uniform no-ARC build is what both platforms' makefiles use.

The easiest path is usually to copy the repo's `Makefile`, rename `DEMO` and `OUT`, and point it at your sources.

3.6 Build for the web (WebAssembly / emscripten)

The same `.m` sources compile unchanged to a browser webapp. You need `brew install emscripten` (the clang-based Wasm toolchain). emscripten ships **no Objective-C runtime and no Foundation**, so the wasm build adds two pieces — both in isolated, gitignored locations, so nothing is installed system-wide and the native `make` build is untouched:

- `.wasm-deps/` — **libobjc2** (the GNUstep ObjC runtime: messaging, classes, memory) cross-compiled for wasm32 with `-fobjc-runtime=gnustep-2.0`, plus three small upstream patches (`wasm/patches/libobjc2-wasm.patch`) that only touch fringe features (ObjC++ exceptions, associated objects, block trampolines — messaging/classes/pools needed none). Built by `wasm/scripts/build-deps.sh`.
- `wasm/MiniFoundation/` — a minimal **Foundation shim** (`NSObject`, `NSString`, `NSConstantString`, `NSMutableString`, `NSArray`, `NSLog`) compiled against that runtime. The wrapper needs ~15 selectors; the full gnustep-base (~204k LOC) has no wasm support in-tree (fork/exec, dlopen, sockets, …) and would add MiBs to the binary.

Commands:

```
make -f Makefile.wasm deps     # cross-build libobjc2 + raylib-web (one-time, slow)
make -f Makefile.wasm test     # MiniFoundation unit tests, run under node
make -f Makefile.wasm demo     # -> wasm/build/demo.html + .js + .wasm + .data
make -f Makefile.wasm run      # build + serve http://localhost:8000/demo.html
```

Then open **http://localhost:8000/demo.html** in a browser. Do **not** open the HTML via `file://` — browsers block the `.wasm` fetch from file URLs ("both async and sync fetching of the wasm failed"); always serve over HTTP (`make -f Makefile.wasm run` does exactly that).

Build details that differ from the native route:

- raylib is compiled for `PLATFORM=Web` (OpenGL ES2) into `vendor/raylib/build-web/` and linked with `-sUSE_GLFW=3 -sASYNCIFY -sALLOW_MEMORY_GROWTH=1` — the web backend drives a synchronous frame loop through `emscripten_sleep()`.
- Sources compile with `emcc -x objective-c -fno-objc-arc -fobjc-runtime=gnustep-2.0 -std=gnu11` (gnu11, not strict c11: `strnlen` is hidden under `__STRICT_ANSI__` on musl).
- The final link uses **em++, not emcc** — `libobjc.a` contains C++ objects and needs libc++.
- The shell (`wasm/shell.html`) sets `Module.canvas` and `arguments: ['/gravnav.rgs']`. Emscripten prepends the program name as `argv[0]` by itself, so the style lands in `argv[1]` — the exact same `argv[1]` code path the native demo uses. The style file is embedded with `--preload-file custom-styles/gravnav.rgs@/gravnav.rgs`, because a browser has no filesystem access at runtime (the pack lands in `demo.data`).

To build *your own* GUI app as a webapp: copy `Makefile.wasm`, point `DEMO_SRC` at your `main.m`, and keep to the Foundation classes the shim provides — extend `wasm/MiniFoundation/` if you need more. The `objc-wasm-emscripten` skill (`~/.vibe/skills/objc-wasm-emscripten/`) has the general recipe and the shim-vs-gnustep-base tradeoff in detail.

---

4. Your first app

Save this as `main.m` and compile it with the commands from §3.5. (This listing is compile-verified — it builds and links clean with `-Wall` through the project's Makefile on macOS.)

```
// main.m — your first RayguiObjC app: a window, a label, a button, a counter.
#import <Foundation/Foundation.h>
#include "raylib.h"        // raylib: window, drawing, input — called directly
#import "RayguiObjC.h"    // RayguiObjC: the GUI controls

int main(int argc, const char *argv[]) {
    @autoreleasepool {
        // --- Setup (once) ---
        InitWindow(480, 320, "My first GUI");
        if (!IsWindowReady()) {
            // No display (headless server, SSH session): raylib logs the
            // GLFW error; exit cleanly instead of crashing in the loop.
            fprintf(stderr, "could not open a window (no display?)\n");
            return 1;
        }
        SetTargetFPS(60);

        // --- UI state: declared ONCE, outside the loop ---
        int clicks = 0;

        // --- Frame loop ---
        while (!WindowShouldClose()) {
            @autoreleasepool {   // per frame: see "Memory" below
                // 1. Update: your app logic goes here (input queries, simulation…)

                // 2. + 3. Draw: BeginDrawing, then one call per control
                BeginDrawing();
                    ClearBackground(RAYWHITE);

                    [RGGui label:RGRectMake(20, 20, 300, 24)
```

```
                                text:@"Hello from RayguiObjC!"];

                    if ([RGGui button:RGRectMake(20, 60, 160, 32) text:@"Click me"])
{
                        clicks++;   // the button reports a click as YES, once
                    }

                    [RGGui label:RGRectMake(20, 108, 300, 24)
                            text:[NSString stringWithFormat:@"Clicked %d times",
clicks]];
                EndDrawing();   // 4.
            }
        }

        // --- Shutdown (once) ---
        CloseWindow();
    }
    return 0;
}
```

Line-by-line:

- `#include "raylib.h"` — the raylib API (`InitWindow`, `BeginDrawing`, `ClearBackground`, `RAYWHITE`, …). Plain C, callable from Objective-C.
- `#import "RayguiObjC.h"` — the wrapper. It includes `raylib.h` itself and declares the `RGGui` facade class, the `RGGResult` enum, and the `RGRectMake`/`RGGColorMake` helpers.
- `@autoreleasepool { … }` — Objective-C without ARC needs a pool for autoreleased objects. **Use one pool per frame, inside the loop** (plus the outer one around `main`'s body for setup-time allocations). Every control call creates autoreleased `NSStrings` (`UTF8String` conversions, item joining); with a single pool around `main` those accumulate for the whole lifetime of the app under manual retain/release. The per-frame pool drains them 60 times a second.
- `InitWindow(480, 320, "My first GUI")` — opens the window. raylib also installs a default exit key: pressing **ESC** makes `WindowShouldClose()` return true (disable with `SetExitKey(0)` if you don't want that).
- `IsWindowReady()` — guard for headless environments.
- `int clicks = 0;` — **this variable is the button's label state.** Because raygui is immediate mode, whatever you want to persist between frames must live in a variable like this.
- `RGRectMake(20, 60, 160, 32)` — a rectangle at x=20, y=60, 160×32 pixels (see §5).
- `[RGGui button:… text:@"Click me"]` — draws the button *and* processes input. Returns `YES` on exactly the frame the user clicks it (mouse button released over the button). The `if` runs your app logic.
- `[NSString stringWithFormat:…]` — regular Foundation; the label shows the counter. Re-evaluated every frame, so it always shows the current value.

Run it, click the button, watch the counter. That is the whole model.

---

5. Coordinates, rectangles, and colors

Screen coordinates are pixels, origin at the top-left corner, y growing downward — the raylib convention, not the Cocoa one.

Every control takes its position and size as a raylib `Rectangle`, a plain C struct passed by value:

```
typedef struct Rectangle { float x, y, width, height; } Rectangle;
```

Build one with the `RGRectMake` helper (mirrors `NSRectMake`):

```
Rectangle r = RGRectMake(20, 60, 160, 32);   // x, y, width, height
[RGGui button:r text:@"Click me"];
```

You can also use raylib's own literals if you prefer C style: `(Rectangle){ 20, 60, 160, 32 }`.

Layout tips:

- `GetScreenWidth()` / `GetScreenHeight()` return the current window size — use them instead of hard-coding, e.g. to pin a status bar to the bottom: `RGRectMake(0, GetScreenHeight() - 24, GetScreenWidth(), 24)`.
- Overlapping controls are drawn in call order (later calls paint on top) and both receive input — keep controls from overlapping.
- For text measurement, `[RGGui textWidth:@"some text"]` returns the width in pixels under the current style/font — handy for centering labels.

Colors are raylib `Color` structs, components 0–255:

```
Color c = RGGColorMake(255, 0, 0, 255);   // r, g, b, a — opaque red
```

raylib's named constants (`RAYWHITE`, `BLANK`, `RED`, `Fade(RAYWHITE, 0.8f)`, …) work anywhere a `Color` is needed — e.g. `ClearBackground(RAYWHITE)`. Colors are passed to controls mainly through the *style* system (§13), or directly to the color-picker controls (`Color *` in/out).

---

6. How a control reports interaction (`RGGResult`)

In raygui 5.0 **every control returns an integer result**. RayguiObjC exposes it as the `RGGResult` enum:

Value	Enum	Meaning
0	`RGGResultNone`	Nothing happened this frame.
1	`RGGResultPressed`	The control was activated — a click, a confirmation, entering/leaving edit mode, a dialog button.
2	`RGGResultChanged`	A bound value changed (slider moved, checkbox toggled, selection changed).
4	`RGGResultTabClose`	A tab bar tab was asked to close — middle-click on the tab, or its close button when the `TABBAR/TAB_CLOSE_BUTTON` style is enabled. The caller should remove the tab from the items array and fix the active index.

Every **interactive** control gets **two methods**, applied uniformly:

- A **`BOOL` convenience** method (`button:`, `checkBox:`, `slider:`, …) that returns `YES` when the control reports an interaction. What exactly "an interaction" means is per-control — see the table below.
- A **`...Raw` variant** (`buttonRaw:`, `checkBoxRaw:`, `sliderRaw:`, …) returning the raw `RGGResult`, for when you need to distinguish *pressed* from *changed* (e.g. a button that must react differently to a click than to a value change) or handle control-specific results.

What `YES` means per control (convenience variant):

Control	**`YES`** means	**`Raw`** useful for
`button:`, `labelButton:`	clicked (`Pressed`)	—
`checkBox:`, `toggle:`	value toggled (`Changed`)	—
`slider:`	value changed (`Changed`)	—
`sliderBar:`	value changed *or* bar clicked (`!= None`)	distinguishing
`toggleGroup:`	selection changed (`Changed`)	—
`toggleSlider:`	selection changed or pressed (`!= None`)	distinguishing
`comboBox:`, `listView:`	selection changed (`Changed`)	—
`dropdownBox:`	opened, closed, or selection changed (`!= None`)	distinguishing
`textBox:`, `valueBox:`, `spinner:`	edit-mode toggled / button pressed (`Pressed`)	—
`valueBoxFloat:`	edit-mode toggled / button pressed (`Pressed`)	—
`dummyRec:`, `grid:`	clicked (`Pressed`)	`grid` also reports the hovered cell via `mouseCell`
`windowBox:`	close button clicked (`Pressed`)	—
`statusBar:`	clicked (`Pressed`)	—
`colorPicker:`, `colorPanel:`, `colorPickerHSV:`	color changed (`!= None`)	distinguishing
`colorPanelHSV:`, `colorBarAlpha:`, `colorBarHue:`	value changed (`Changed`)	—

The rule of thumb: **use the `BOOL` variant 95% of the time**; reach for the `...Raw` variant when `Pressed` vs `Changed` matters to you.

Exceptions to the two-method rule: the purely visual controls — `label:`, `groupBox:`, `panel:`, `line:`, `progressBar:` — return `void` and have no `Raw` variant; `scrollPanel:` returns the visible `Rectangle` instead of a result.

Controls that return something other than `BOOL`/`RGGResult`:

- `scrollPanel:` returns the visible `Rectangle` (§12).
- `messageBox:` / `textInputDialog:` return the **1-based index of the pressed button**, `0` if none (§11).
- `tabBar:` returns the **active tab index**, clamped to the item range (–1 only when the item list is empty). It cannot report a tab-close request — use `tabBarRaw:` for that.

---

7. The bound-state pattern

Controls that display or edit a value don't store it — **you** store it and pass a C pointer. The control reads the value to draw, and writes the new value back if the user interacts:

```
BOOL  enabled  = YES;     // your state
float volume   = 0.5f;
int   quality  = 0;       // index into the items array
int   selected = -1;      // -1 = no selection (list views)

while (!WindowShouldClose()) {
    BeginDrawing();

        [RGGui checkBox:RGRectMake(20, 20, 24, 24) text:@"Enabled"
checked:&enabled];
        [RGGui slider:RGRectMake(20, 56, 200, 24) text:@"Volume"
                value:&volume minValue:0.0f maxValue:1.0f];
        [RGGui comboBox:RGRectMake(20, 92, 160, 28)
                   items:@[@"Low", @"Medium", @"High"] selected:&quality];

        // …and later in your app logic, just read the variables:
        // if (enabled) playSound(volume);

    EndDrawing();
}
```

Notes:

- Declare the variables **once, outside the loop**. A variable created inside the loop resets every frame — the classic immediate-mode bug (see §16).
- Pointers are plain C pointers (`&enabled`, `&volume`) — no `NSNumber` wrapping in the per-frame path.
- For `BOOL` state, pass `BOOL *`; the wrapper converts to/from raygui's C `bool` internally.
- Lists take an `NSArray<NSString *> *`; the wrapper joins the items with `;` for you (raygui's convention). An empty array is fine.

---

8. Strings, arrays, and memory rules

Display text is `NSString *`. It is converted to a C string with `UTF8String` for the duration of the call:

```
[RGGui label:RGRectMake(20, 20, 200, 24) text:@"Plain text"];
[RGGui label:RGRectMake(20, 20, 200, 24)
        text:[NSString stringWithFormat:@"Volume: %.0f%%", volume * 100]];
```

Never store the pointer returned by `UTF8String` — it is only valid until the autorelease pool drains (the wrapper never does this; it copies or uses it within the call).

Editable text is `NSMutableString *`. The control writes the edited text back into it every frame:

```
NSMutableString *name = [NSMutableString stringWithString:@"Player 1"];
// per frame:
[RGGui textBox:RGRectMake(20, 20, 200, 30) text:name maxLength:64];
// name now holds whatever the user typed
```

`maxLength` caps the text length in **UTF-8 bytes, not characters** — a two-byte character (é, 漢, …) counts as two, a three-byte one as three. It is clamped internally to `RGGTextMaxLength` (1023) — the wrapper copies through a fixed 1024-byte stack buffer (`RGGTextBufferSize`), so stay at or below that. Two edge cases to know:

- raygui never splits a multibyte character while *typing* (it checks that the whole encoded character fits). The wrapper also protects the other direction: its copy into the C buffer is clamped at a UTF-8 boundary (a multibyte character is never split), and the write-back guards against invalid UTF-8 (an undecodable buffer leaves your string unchanged instead of throwing). So even an initial string longer than 1023 bytes is truncated safely — but keep it well below the limit anyway.
- See §10 for the `editMode` variant that actually enables typing.

The two helpers behind this are public (`RGGCopyToBuffer` and `RGGSetStringFromBuffer`, alongside `RGGJoinItems`) — exposed for testing, and usable directly if you ever need the same NSString↔C-string bridging.

Arrays (`NSArray<NSString *> *`) are used for every multi-item control — combo box, dropdown box, list view, toggle group, toggle slider, tab bar, dialog buttons. The wrapper joins them with `;` per raygui convention (the public helper `RGGJoinItems()` does the same join if you ever need it). Two consequences:

- An item containing `;` (e.g. `@"a;b"`) **silently splits into two items** — `;` is the separator and is not escapable. Keep it out of item text.
- Item text may contain icon markers (`@"#18#Squirtle"` — see §13.4); they pass through unmodified.
- raygui splits the joined string into a static buffer holding at most **128 items / 1023 bytes**; longer lists are silently truncated. The wrapper logs a `WARNING` when your list exceeds either limit — if you see it, shorten the list or the item texts.

Memory: the project is manual retain/release (no ARC). Create `NSMutableStrings` once outside the loop and reuse them. Inside the loop, the only allocations are the autoreleased `NSString` conversions the wrapper makes per call (`UTF8String` bridges, item joining, `stringWithFormat:` in your own code). Under manual retain/release those accumulate until a pool drains — so **use a per-frame `@autoreleasepool` inside the loop** (see §4), not just the one around `main`'s body.

---

9. Controls reference

All methods are **class methods on `RGGui`** — you never instantiate it: `[RGGui button:…]`, not `[[RGGui alloc] init]`.

9.1 Basic controls — `RGGui (Controls)`

Label — static text, never interacts:

```
+ (void)label:(Rectangle)bounds text:(NSString *)text;
```

Button — YES on click:

```
+ (BOOL)button:(Rectangle)bounds text:(NSString *)text;
+ (RGGResult)buttonRaw:(Rectangle)bounds text:(NSString *)text;
```

Label button — a clickable text link, same semantics as a button:

```
+ (BOOL)labelButton:(Rectangle)bounds text:(NSString *)text;
+ (RGGResult)labelButtonRaw:(Rectangle)bounds text:(NSString *)text;
```

Toggle — a button that stays pressed. `active` is in/out:

```
+ (BOOL)toggle:(Rectangle)bounds text:(NSString *)text active:(BOOL *)active;
+ (RGGResult)toggleRaw:(Rectangle)bounds text:(NSString *)text active:(BOOL
*)active;
```

Toggle group — a row of mutually exclusive toggle buttons. `items` is the `NSArray` of labels; `active` is the in/out index of the selected item:

```
+ (BOOL)toggleGroup:(Rectangle)bounds items:(NSArray<NSString *> *)items active:(int
*)active;
```

Toggle slider — a segmented slider: one bar, one selected segment (styled as `SLIDER`, not `TOGGLE`). Same parameters:

```
+ (BOOL)toggleSlider:(Rectangle)bounds items:(NSArray<NSString *> *)items active:
(int *)active;
```

Check box:

```
+ (BOOL)checkBox:(Rectangle)bounds text:(NSString *)text checked:(BOOL *)checked;
+ (RGGResult)checkBoxRaw:(Rectangle)bounds text:(NSString *)text checked:(BOOL
*)checked;
```

Slider / slider bar — `value` is in/out, clamped to `[minValue, maxValue]`:

```
+ (BOOL)slider:(Rectangle)bounds text:(NSString *)text
         value:(float *)value minValue:(float)min maxValue:(float)max;
+ (BOOL)sliderBar:(Rectangle)bounds text:(NSString *)text
            value:(float *)value minValue:(float)min maxValue:(float)max;
```

The label is drawn **to the left of the bar, outside its bounds** — leave room for it, or pass `nil` as `text` and draw your own label. raygui 5.0 draws no value text by itself, and the wrapper passes `NULL` for raygui's right-hand label — show the current value with your own `label:` (the demo does exactly that).

Progress bar — display-only bar for a value (e.g. download progress). Returns `void`; the value is passed **by value** (AGENTS.md §3.1). raygui draws no text on it — use your own `label:` if you want one:

```
+ (void)progressBar:(Rectangle)bounds value:(float)value minValue:(float)min
maxValue:(float)max;
```

Text box — editable single-line text bound to an `NSMutableString`:

```
+ (BOOL)textBox:(Rectangle)bounds text:(NSMutableString *)text maxLength:
(int)maxLength;
+ (BOOL)textBox:(Rectangle)bounds text:(NSMutableString *)text maxLength:
(int)maxLength editMode:(BOOL)editMode;
+ (RGGResult)textBoxRaw:(Rectangle)bounds text:(NSMutableString *)text maxLength:
(int)maxLength editMode:(BOOL)editMode;
```

Use the `editMode:` variant and toggle your own flag — see §10. The short `textBox:…maxLength:` variant is **display-only**: it shows the text and reports clicks, but never accepts typing.

The wrapper passes `NULL` for raygui's label parameter on the numeric controls below, so those labels are not drawn — add your own `label:` if you want one.

Value box / value box float — a numeric field with −/+ or direct typing. `valueBoxFloat:` also binds the text representation (`textValue`):

```
+ (BOOL)valueBox:(Rectangle)bounds value:(int *)value minValue:(int)min maxValue:
(int)max;
+ (BOOL)valueBoxFloat:(Rectangle)bounds textValue:(NSMutableString *)textValue
value:(float *)value;
```

Both have `editMode:` and `...Raw` variants like `textBox`. The short (no-`editMode`) variants are **display-only** — without edit mode these controls accept no keyboard input.

Combo box — a drop-down selection. `selected` is the in/out 0-based index:

```
+ (BOOL)comboBox:(Rectangle)bounds items:(NSArray<NSString *> *)items selected:(int
*)selected;
+ (RGGResult)comboBoxRaw:(Rectangle)bounds items:(NSArray<NSString *> *)items
selected:(int *)selected;
```

Dropdown box — like a combo box with an explicit open/close state. Use the `editMode:` variant with your own flag (§10) — the short variant pins `editMode` to `NO`, so **the list can never open**; it is display-only:

```
+ (BOOL)dropdownBox:(Rectangle)bounds items:(NSArray<NSString *> *)items selected:
(int *)selected;
+ (BOOL)dropdownBox:(Rectangle)bounds items:(NSArray<NSString *> *)items selected:
(int *)selected editMode:(BOOL)editMode;
+ (RGGResult)dropdownBoxRaw:(Rectangle)bounds items:(NSArray<NSString *> *)items
selected:(int *)selected editMode:(BOOL)editMode;
```

Spinner — a numeric field with −/+ buttons. The arrow buttons work without edit mode; the `editMode:` variant additionally allows typing the value:

```
+ (BOOL)spinner:(Rectangle)bounds value:(int *)value minValue:(int)min maxValue:
(int)max;
+ (BOOL)spinner:(Rectangle)bounds value:(int *)value minValue:(int)min maxValue:
(int)max editMode:(BOOL)editMode;
+ (RGGResult)spinnerRaw:(Rectangle)bounds value:(int *)value minValue:(int)min
maxValue:(int)max editMode:(BOOL)editMode;
```

Dummy rectangle — an invisible clickable placeholder (useful for custom drawn areas):

```
+ (BOOL)dummyRec:(Rectangle)bounds text:(NSString *)text;
```

Grid — a visible grid, e.g. for level editors. `mouseCell` receives the currently hovered cell as a `Vector2` (column, row):

```
+ (BOOL)grid:(Rectangle)bounds spacing:(float)spacing subdivs:(int)subdivs
mouseCell:(Vector2 *)mouseCell;
```

9.2 Containers — `RGGui (Containers)`

Window box — a closable framed window with a title bar. YES when its close button is clicked. It is **not draggable** — raygui draws the frame and close button only; dragging is the app's job (see raygui's own `floating_window` example for a manual implementation):

```
+ (BOOL)windowBox:(Rectangle)bounds title:(NSString *)title;
+ (RGGResult)windowBoxRaw:(Rectangle)bounds title:(NSString *)title;
```

Group box / panel / line — static visual grouping (never interact):

```
+ (void)groupBox:(Rectangle)bounds text:(NSString *)text;   // framed group with
label
+ (void)panel:(Rectangle)bounds text:(NSString *)text;      // filled panel
+ (void)line:(Rectangle)bounds text:(NSString *)text;       // separator, optional
text
```

Scroll panel — a clipped, scrollable region. See §12 for the full pattern.

```
+ (Rectangle)scrollPanel:(Rectangle)bounds content:(Rectangle)content scroll:
(Vector2 *)scroll;
+ (Rectangle)scrollPanel:(Rectangle)bounds text:(NSString *)text content:
(Rectangle)content scroll:(Vector2 *)scroll;
```

Status bar — an informational bar, typically pinned to the bottom:

```
+ (BOOL)statusBar:(Rectangle)bounds text:(NSString *)text;
```

9.3 Advanced controls — `RGGui (Advanced)`

List view — a scrollable single-selection list. Initialize `selected` to –1 for "no selection" and `scroll` (the scroll index) to `0`:

```
+ (BOOL)listView:(Rectangle)bounds items:(NSArray<NSString *> *)items
        selected:(int *)selected scrollIndex:(int *)scroll;
+ (RGGResult)listViewRaw:(Rectangle)bounds items:(NSArray<NSString *> *)items
               selected:(int *)selected scrollIndex:(int *)scroll;
```

Color controls — all take a pointer to a `Color` (or `Vector3` HSV) that is updated in place:

```
+ (BOOL)colorPicker:(Rectangle)bounds color:(Color *)color;          // full picker
(wheel + alpha)
+ (BOOL)colorPanel:(Rectangle)bounds color:(Color *)color;           //
saturation/value square
+ (BOOL)colorBarAlpha:(Rectangle)bounds alpha:(float *)alpha;        // alpha
slider, 0.0–1.0
+ (BOOL)colorBarHue:(Rectangle)bounds hue:(float *)hue;              // hue slider,
0–360
+ (BOOL)colorPickerHSV:(Rectangle)bounds colorHsv:(Vector3 *)colorHsv;
+ (BOOL)colorPanelHSV:(Rectangle)bounds colorHsv:(Vector3 *)colorHsv;
```

Each has a `...Raw` variant. Note: raygui 5.0 as pinned here includes the upstream fix for the disappearing hue bar in `colorPicker:` (see `VERSIONS.md`).

Message box and **text input dialog** — modal dialogs. See §11.

```
+ (int)messageBox:(Rectangle)bounds title:(NSString *)title message:(NSString
*)message
          buttons:(NSArray<NSString *> *)buttons;
+ (RGGResult)messageBoxRaw:(Rectangle)bounds title:(NSString *)title message:
(NSString *)message
                  buttons:(NSArray<NSString *> *)buttons pressedButton:(int
*)pressedButton;
```

```
+ (int)textInputDialog:(Rectangle)bounds title:(NSString *)title message:(NSString
*)message
                buttons:(NSArray<NSString *> *)buttons text:(NSMutableString *)text;
+ (int)textInputDialog:(Rectangle)bounds title:(NSString *)title message:(NSString
*)message
                buttons:(NSArray<NSString *> *)buttons text:(NSMutableString *)text
             secretView:(BOOL *)secretView;
```

Tab bar — a row of tabs. The convenience variant takes the current tab by value and returns the (possibly new) active index, clamped to the item range (−1 only when the item list is empty):

```
+ (int)tabBar:(Rectangle)bounds items:(NSArray<NSString *> *)items active:
(int)active;
+ (RGGResult)tabBarRaw:(Rectangle)bounds items:(NSArray<NSString *> *)items active:
(int *)active;
```

With the Raw variant, `RGGResultChanged` means the user switched tabs and `RGGResultTabClose` means a tab was asked to close — middle-click on the tab, or its close button when the `RGGTabCloseButton` style property is enabled (`[RGGui setStyle:RGGControlTabBar property:RGGTabCloseButton value:1]`). On a close request the caller should remove that tab from the items array and fix the active index. The convenience `tabBar:` returns only the index, so it cannot report a close — use `tabBarRaw:` if your tabs are closable.

9.4 Global state and style — `RGGui (State)`

```
+ (void)enable;                          // controls accept input again
+ (void)disable;                         // all controls greyed out, no input
+ (void)lock;                            // controls can&#39;t be interacted with
+ (void)unlock;
+ (BOOL)isLocked;

+ (void)setAlpha:(float)alpha;           // global control transparency, 0.0–1.0
+ (void)setState:(RGGState)state;        // force Normal/Focused/Pressed/Disabled
+ (RGGState)state;

+ (void)setFont:(Font)font;              // raylib Font, e.g. from LoadFont()
+ (Font)font;

+ (void)setStyle:(RGGControl)control property:(RGGProperty)property value:
(int)value;
+ (int)style:(RGGControl)control property:(RGGProperty)property;

+ (BOOL)loadStyleAtPath:(NSString *)path;   // load a .rgs theme file
+ (void)loadStyleDefault;                   // back to the built-in theme

+ (void)enableTooltip;
+ (void)disableTooltip;
+ (void)setTooltip:(NSString *)tooltip;  // tooltip shown for the focused control

+ (NSString *)iconText:(int)iconId text:(NSString *)text;  // @"#001#text" helper
+ (void)setIconScale:(int)scale;

+ (int)textWidth:(NSString *)text;       // pixel width under current style
```

The `RGGControl` enum names the style target (`RGGControlButton`, `RGGControlSlider`, `RGGControlDefault` for global, …); `RGGProperty` names the property (`RGGBaseColorNormal`, `RGGTextSize`, `RGGBorderWidth`, …). Color properties take a **packed RGBA int in raylib's `0xRRGGBBAA` format** — pass the literal directly; raygui applies `GetColor()` to it internally. (raylib's

`GetColor()` goes the other way, int → `Color`; `ColorToInt()` converts a `Color` back to the packed int.) Full enums are in `src/RayguiObjC.h`; §13 shows practical styling.

`setState:` forces one visual state (`RGGStateNormal`/`Focused`/`Pressed`/ `Disabled`) on every control drawn after it, until the next `setState:` — raygui's own test suite uses it to preview each state's look. Rarely needed in app code; `state` reads the current global state back.

9.5 Beyond the wrapper: calling raygui and raylib directly

The wrapper covers the 35 raygui controls plus the state/style API, but raygui offers a few more public functions it does not wrap:

- `GuiLoadStyleFromMemory()` — load a binary style from memory
- `GuiGetIcons()` / `GuiLoadIcons()` / `GuiLoadIconsFromMemory()` — custom icon sets (`.rgi`)
- `GuiDrawIcon()` — draw a single icon anywhere
- `GuiListViewEx()` / `GuiTabBarEx()` — array-based variants that also report the focused item

You can call these (or any other raygui function) directly: add `#include "raygui.h"` to your `.m` file, **after** `#include "raylib.h"`. Without `RAYGUI_IMPLEMENTATION` the header is declarations only — the implementation is already compiled into the wrapper's `src/RayguiImplementation.o`, so you get no duplicate symbols.

And raylib itself is never wrapped — every raylib function (`DrawText`, `LoadFont`, `BeginScissorMode`, `GetColor`, …) is called directly all the time. If a wrapper method is ever missing, the escape hatch is always the underlying C function.

---

10. Editable text: the `editMode` pattern

Text-entry controls (`textBox:`, `valueBox:`, `valueBoxFloat:`, `spinner:`, `dropdownBox:`) work in two modes:

- **Not in edit mode:** the control displays its value; clicking it returns `YES` (`RGGResultPressed`).
- **In edit mode:** the control shows a cursor and accepts keyboard input; pressing ENTER or clicking outside ends editing and returns `YES` again.

The control never stores the edit-mode flag — **you do**, and you toggle it when the call returns `YES`:

```
NSMutableString *name = [NSMutableString stringWithString:@"Player 1"];
BOOL nameEditMode = NO;

// per frame:
if ([RGGui textBox:RGRectMake(20, 60, 260, 30)
              text:name maxLength:64 editMode:nameEditMode]) {
    nameEditMode = !nameEditMode;   // clicked in → edit; ENTER/click out → stop
}
```

The same pattern applies to `valueBox:…editMode:`, `valueBoxFloat:…editMode:`, `spinner:…editMode:`, and `dropdownBox:…editMode:` (for the dropdown, the flag is "the list is open").

Gotcha: the short `textBox:…maxLength:` variant passes `editMode:NO` permanently — the box displays and reports clicks, but **typing never works**. Always use the `editMode:` variant for editable text.

`maxLength` caps the text length in UTF-8 bytes (see §8) and is clamped to `RGGTextMaxLength` = 1023.

---

11. Dialogs: message boxes and text input

Dialogs are just controls you call conditionally. The standard pattern: a `BOOL` "dialog is open" flag, make the background controls inert, dim the background, call the dialog, close it when the user dismisses it.

Dialogs are not modal by themselves. A dimmed background does not stop the controls underneath from receiving clicks and keystrokes. raygui's way to make a dialog modal is the global lock: while locked, every control still draws but ignores input. Lock *before* the background controls, unlock right before the dialog (so the dialog itself stays interactive) — this is the pattern raygui's own test suite uses for its text-input dialog:

```
BOOL showSaveDialog = NO;
int  saveChoice = 0;   // remembers what the user picked

// per frame:
if (showSaveDialog) [RGGui lock];       // background controls become draw-only

// ... all your regular controls ...
if ([RGGui button:RGRectMake(20, 400, 120, 30) text:@"Save settings"]) {
    showSaveDialog = YES;
}

if (showSaveDialog) [RGGui unlock];     // the dialog itself stays interactive

if (showSaveDialog) {
    // dim everything behind the dialog
    DrawRectangle(0, 0, GetScreenWidth(), GetScreenHeight(), Fade(RAYWHITE, 0.8f));

    int btn = 0;
    RGGResult r = [RGGui messageBoxRaw:RGRectMake(170, 160, 300, 120)
                                 title:@"Save"
                               message:@"Save current settings?"
                               buttons:@[@"Yes", @"No"]
                         pressedButton:&btn];
    if (r == RGGResultPressed) {   // any dismissal: a button, or the X
        showSaveDialog = NO;
        saveChoice = btn;          // 0 = X, 1 = "Yes", 2 = "No"
        NSLog(@"Save choice: %d", saveChoice);
    }
}
```

Return-value details:

- The `Raw` variant returns `RGGResultPressed` on *any* dismissal and reports which through `pressedButton`: the **1-based index** into your `buttons` array (`1` = first button), or `0` when the dialog's close (X) button was pressed.

- The convenience `messageBox:` returns just that index — `0` both when nothing was clicked *and* when X was pressed, so with the convenience variant **the X button appears dead** (the dialog can never be dismissed that way). Use the `Raw` variant whenever the dialog must be dismissible via X.

Text input dialog — same pattern, plus a bound `NSMutableString` and an optional password mode:

```
NSMutableString *userName = [NSMutableString stringWithString:@""];
BOOL showNameDialog = NO;
BOOL secret = NO;   // pass &secret to mask the input (password mode)

// per frame:
if (showNameDialog) [RGGui lock];
// ... background controls ...
if (showNameDialog) [RGGui unlock];

if (showNameDialog) {
    DrawRectangle(0, 0, GetScreenWidth(), GetScreenHeight(), Fade(RAYWHITE, 0.8f));
    int btn = 0;
    RGGResult r = [RGGui textInputDialogRaw:RGRectMake(170, 150, 300, 140)
                                      title:@"Profile"
                                    message:@"Enter your name:"
                                    buttons:@[@"OK", @"Cancel"]
                                       text:userName
                                 secretView:&secret
                              pressedButton:&btn];
    if (r == RGGResultPressed) {
        showNameDialog = NO;
        if (btn == 1) NSLog(@"Hello, %@", userName);
    }
}
```

The dialog's text is capped at `RGGTextMaxLength` (1023) bytes (see §8). Only one text-entry dialog should be open at a time (raygui keeps internal edit state for it).

---

12. Scrollable areas

`scrollPanel:` clips a large content area into a smaller panel with scroll bars. It returns the **view rectangle** — the visible area in screen coordinates — and updates `scroll` (a `Vector2`, in/out) with the current scroll offset.

The contract (mirrors raygui's own `scroll_panel` example):

1. Call `scrollPanel:` with the panel bounds and the **full content size**.
2. Wrap the child controls in raylib's `BeginScissorMode(view)` / `EndScissorMode()` so children are clipped to the visible area.
3. Position children at **panel origin + scroll offset + child position**.

```
Rectangle panel   = RGRectMake(20, 60, 220, 300);
Rectangle content = RGRectMake(0, 0, 400, 800);   // full scrollable size
Vector2 scroll    = { 0, 0 };                     // persists across frames

// per frame:
Rectangle view = [RGGui scrollPanel:panel content:content scroll:&scroll];
```

```
BeginScissorMode((int)view.x, (int)view.y, (int)view.width, (int)view.height);
    for (int i = 0; i < 20; i++) {
        [RGGui labelButton:RGRectMake(panel.x + scroll.x + 10,
                                      panel.y + scroll.y + 10 + i*28,
                                      200, 24)
                      text:[NSString stringWithFormat:@"Row %d", i]];
    }
EndScissorMode();
```

Notes:

- Keep `scroll` in a variable outside the loop; the panel writes the new offset back into it.
- The `content` rectangle's `width`/`height` drive the scroll bars; its `x`/`y` are conventionally `0`.
- The optional `text:` variant draws a title bar on the panel (the content area then starts below it).
- `BeginScissorMode`/`EndScissorMode` are **raylib** calls (not wrapped) — they take `int` x, y, width, height.

---

13. Styling: themes, colors, fonts, icons, tooltips

13.1 Theme files (`.rgs`)

The fastest way to change the look: load a theme file at startup, before the loop:

```
if (![RGGui loadStyleAtPath:@"vendor/raygui/styles/cyber/style_cyber.rgs"]) {
    NSLog(@"style not found — using default theme");
}
// …or go back to the built-in look at any time:
[RGGui loadStyleDefault];
```

- raygui ships ~18 themes in `vendor/raygui/styles/` (`cyber`, `dark`, `candy`, `terminal`, …), each with a screenshot.
- **Binary `.rgs`** files embed the font and load standalone.
- **Text `.rgs`** files reference an external font/charset — the font file must sit next to the `.rgs` (raygui resolves it relative to the style file).
- `loadStyleAtPath:` returns `NO` if the file can't be opened; raygui itself would only log a message, so the wrapper checks first.
- **In the wasm build there is no runtime filesystem**: assets must be embedded at link time (`--preload-file …@/path`) and `argv[1]` is set from `Module.arguments` in `wasm/shell.html`. The wasm build embeds `custom-styles/gravnav.rgs` automatically — see §3.6.

13.2 Tweaking individual properties at runtime

Every visual property of every control is an integer in a global style table:

```
// All buttons: red background when focused. Style colors are packed ints in
// raylib&#39;s 0xRRGGBBAA format (raygui applies GetColor() to them internally):
[RGGui setStyle:RGGControlButton property:RGGBaseColorFocused value:0xCC2222FF];
```

```objc
// Global default text size (affects all controls unless overridden)
[RGGui setStyle:RGGControlDefault property:RGGTextSize value:20];

// Read a property back
int size = [RGGui style:RGGControlDefault property:RGGTextSize];
```

The full `RGGControl` (which control) and `RGGProperty` (which property) enums are in `src/RayguiObjC.h`. Alignment properties use 0/1/2 (left/center/right, top/middle/bottom).

Because per-control properties share the same numeric slots (16, 17, …), a value's meaning depends on the control you set it on:

Property (value)	Applies to (`RGGControl`)	Meaning
`RGGBorderColorNormal` … `RGGTextColorDisabled` (0–11)	every control	border/base/text color for state Normal/Focused/Pressed/Disabled
`RGGBorderWidth` (12), `RGGTextPadding` (13), `RGGTextAlignment` (14)	every control	border size, text padding, horizontal alignment (0/1/2)
`RGGTextSize` (16), `RGGTextSpacing` (17), `RGGLineColor` (18), `RGGBackgroundColor` (19), `RGGTextLineSpacing` (20), `RGGTextAlignVertical` (21), `RGGTextWrapMode` (22)	`RGGControlDefault` only	global text/background properties
`RGGGroupPadding` (16), `RGGGroupWidthFull` (17)	`RGGControlToggle`	toggle-group item spacing / full-width items
`RGGSliderWidth` (16), `RGGSliderPadding` (17)	`RGGControlSlider`	slider bar width / padding
`RGGProgressPadding` (16), `RGGProgressSide` (17)	`RGGControlProgressBar`	progress padding / fill side
`RGGCheckPadding` (16)	`RGGControlCheckBox`	check-mark padding
`RGGComboButtonWidth` (16), `RGGComboButtonSpacing` (17)	`RGGControlComboBox`	drop-down button width / spacing
`RGGArrowPadding` (16), `RGGDropdownItemsSpacing` (17), `RGGDropdownArrowHidden` (18), `RGGDropdownRollUp` (19)	`RGGControlDropdownBox`	arrow padding / item spacing / hide arrow / roll up
`RGGTextReadonly` (16)	`RGGControlTextBox`	1 = read-only text box
`RGGSpinnerButtonWidth` (16), `RGGSpinnerButtonSpacing` (17)	`RGGControlValueBox`	spinner/value-box button width / spacing
`RGGTabItemsWidth` (16), `RGGTabCloseButton` (17), `RGGTabLineSide` (18)	`RGGControlTabBar`	tab width / show close buttons / line side (0-bottom, 1-top)
`RGGListItemsHeight` (16), `RGGListItemsSpacing` (17), `RGGListItemsBorderNormal` (20), `RGGListItemsBorderWidth` (21)	`RGGControlListView`	item height / spacing / border

Property (value)	Applies to (`RGGControl`)	Meaning
`RGGScrollBarWidth` (18), `RGGScrollBarSide` (19)	`RGGControlListView`	scrollbar width / side (0=left, 1=right)
`RGGScrollSpeed` (21)	`RGGControlScrollBar`	scroll speed
`RGGColorSelectorSize` (16), `RGGColorHueBarWidth` (17), `RGGColorHueBarPadding` (18), `RGGColorHueBarSelectorHeight` (19), `RGGColorHueBarSelectorOverflow` (20)	`RGGControlColorPicker`	picker selector / hue-bar geometry

The `RGGProperty` enum covers all of raygui 5.0's per-control properties, and every value is pinned to the upstream constant by a compile-time `_Static_assert` in `src/RayguiImplementation.m` — so it cannot silently drift if the pinned raygui ever changes. The full upstream list is in `vendor/raygui/src/raygui.h` (`GuiControlProperty` and the per-control property enums) if you ever need a value the wrapper does not name.

13.3 Custom fonts

```
Font font = LoadFont("vendor/raygui/styles/cyber/Kyrou7Wide.ttf");  // raylib call
[RGGui setFont:font];
// …
[RGGui setFont:[RGGui font]];   // read the current font back
```

When you load a style file that embeds a font, raygui sets the font itself.

13.4 Icons in labels

Any control's text may contain an icon marker of the form `#nnn#` (three-digit icon id 0–255, then the text): `@"#001#Save"` draws icon 1 followed by "Save". raygui's default icon set is embedded; scale it with `[RGGui setIconScale:2]`.

The `iconText:text:` helper builds the marker for you:

```
[RGGui button:bounds text:[RGGui iconText:1 text:@"Save"]];   // → @"#001#Save"
```

13.5 Tooltips

raygui stores a **single global tooltip string**: any focused (hovered) control draws whatever tooltip is currently set, and it is never cleared automatically. So set the tooltip *before* the control it describes, and clear it right after — otherwise it lags one frame, and then sticks to every later control that doesn't set its own:

```
[RGGui enableTooltip];
// per frame:
[RGGui setTooltip:@"Turns the thing on and off"];
[RGGui checkBox:bounds text:@"Enabled" checked:&enabled];
[RGGui setTooltip:nil];    // don&#39;t let it leak to the next focused control
```

The wrapper copies the string into its own buffer before handing it to raygui (raygui stores the pointer globally), so any `NSString` works — including autoreleased ones created inside the frame. `[RGGui disableTooltip]` turns the feature off globally.

13.6 Global switches

```
[RGGui disable];        // grey out the whole GUI (e.g. while a modal is up)
[RGGui enable];
[RGGui lock];           // controls visible but not interactive
[RGGui unlock];
[RGGui setAlpha:0.7f];  // make the whole GUI translucent (fade effects)
```

---

14. Complete example: a settings window

A full app combining everything above: a window box containing an editable text box, a slider with a live value readout, a combo box, a checkbox; a Save button that opens a modal message box; a status bar. Save as `main.m` and build per §3.5. (This listing is compile-verified — it builds and links clean with `-Wall` through the project's Makefile on macOS.)

```
// main.m — settings window: windowBox, textBox, slider, comboBox, checkBox,
// messageBox, statusBar.
#import <Foundation/Foundation.h>
#include "raylib.h"
#import "RayguiObjC.h"

int main(int argc, const char *argv[]) {
    @autoreleasepool {
        const int screenWidth = 480;
        const int screenHeight = 360;

        InitWindow(screenWidth, screenHeight, "Settings");
        if (!IsWindowReady()) return 1;
        SetTargetFPS(60);

        // Optional theme, e.g.: ./settings vendor/raygui/styles/dark/style_dark.rgs
        if (argc > 1) {
            [RGGui loadStyleAtPath:[NSString stringWithUTF8String:argv[1]]];
        }

        // ---- UI state (declared once) ----
        NSMutableString *playerName = [NSMutableString stringWithString:@"Player
1"];
        BOOL nameEditMode = NO;
        float volume = 0.8f;
        int  quality = 1;                       // 0=Low 1=Medium 2=High
        BOOL muted = NO;
        BOOL showSaveDialog = NO;
        int  saveChoice = 0;

        NSArray *qualityItems = @[@"Low", @"Medium", @"High"];

        while (!WindowShouldClose()) {
            @autoreleasepool {
                // ---- Update ----

                // ---- Draw ----
                BeginDrawing();
                    ClearBackground(RAYWHITE);

                    // Modal dialog: background controls draw but take no input
                    if (showSaveDialog) [RGGui lock];

                    if ([RGGui windowBox:RGRectMake(20, 20, 440, 280)
title:@"Settings"]) {
                        // window close button clicked — decorative here
```

```
                    }

                    [RGGui label:RGRectMake(40, 64, 80, 24) text:@"Name:"];
                    if ([RGGui textBox:RGRectMake(130, 60, 200, 30)
                                  text:playerName maxLength:32
editMode:nameEditMode]) {
                        nameEditMode = !nameEditMode;
                    }

                    [RGGui label:RGRectMake(40, 104, 80, 24) text:@"Volume:"];
                    [RGGui slider:RGRectMake(130, 104, 180, 24) text:nil
                           value:&volume minValue:0.0f maxValue:1.0f];
                    [RGGui label:RGRectMake(320, 104, 50, 24)
                           text:[NSString stringWithFormat:@"%.0f%%", volume *
100]];
                    [RGGui checkBox:RGRectMake(380, 104, 24, 24) text:@"Mute"
checked:&muted];

                    [RGGui label:RGRectMake(40, 140, 80, 24) text:@"Quality:"];
                    [RGGui comboBox:RGRectMake(130, 136, 160, 28)
                              items:qualityItems selected:&quality];

                    if ([RGGui button:RGRectMake(130, 180, 120, 30) text:@"Save"]) {
                        showSaveDialog = YES;
                    }

                    [RGGui statusBar:RGRectMake(0, (float)screenHeight - 24,
                                                (float)screenWidth, 24)
                              text:[NSString stringWithFormat:
                                    @"name=%@ volume=%.2f quality=%d muted=%d",
                                    playerName, volume, quality, muted]];

                    if (showSaveDialog) [RGGui unlock];

                    // ---- Modal dialog ----
                    if (showSaveDialog) {
                        DrawRectangle(0, 0, screenWidth, screenHeight,
Fade(RAYWHITE, 0.8f));
                        int btn = 0;
                        RGGResult r = [RGGui messageBoxRaw:RGRectMake(90, 120, 300,
120)
                                                     title:@"Save"
                                                   message:@"Save current
settings?"
                                                   buttons:@[@"Yes", @"No"]
                                             pressedButton:&btn];
                        if (r == RGGResultPressed) {
                            showSaveDialog = NO;
                            saveChoice = btn;   // 0 = X, 1 = "Yes", 2 = "No"
                            NSLog(@"Save choice: %d", saveChoice);
                        }
                    }
                EndDrawing();
            }
        }

        CloseWindow();
    }
    return 0;
}
```

Things to notice:

- All state lives in variables declared before the loop; the loop only reads and writes them.
- The frame body is wrapped in a per-frame `@autoreleasepool` (§4, §8).
- The text box uses the `editMode:` variant with a toggled flag (§10).
- The slider passes `nil` text and uses two explicit labels — raygui draws the built-in label to the left of the bar and no value text (§9.1).
- The dialog is modal: `lock` before the background controls, `unlock` before the dialog, and the `Raw` variant so the X button works (§11).
- The status bar is pinned to the window bottom with the screen height baked into the rectangle.
- Everything, including the dialog, is inside `BeginDrawing()/EndDrawing()`.

---

15. Platform notes

- **Same source, both platforms.** The wrapper uses Foundation only — no AppKit types — so your `.m` files compile unchanged on macOS (Cocoa) and Linux (GNUstep). The Makefile picks flags via `uname -s`.
- **No ARC on either platform.** Compile with `-fno-objc-arc` (the Makefile does). ARC is deliberately not used so the same code builds under GNUstep.
- **Linux display.** The app needs an X11 (or compatible) display. On a headless box, `InitWindow` fails, `IsWindowReady()` returns false, and the demo/examples exit cleanly — that guard is deliberate, keep it in your apps.
- **Linux prerequisites** are listed in §3.1; the link line needs X11/GL and the GNUstep base libs (`gnustep-config --base-libs`).
- **macOS frameworks** at link time: `Foundation`, `Cocoa`, `IOKit`, `CoreFoundation`, `OpenGL` (see §3.5).
- **Web (Wasm).** The app runs in a browser tab, not an X11 window — see §3.6. No filesystem at runtime: style/font assets are embedded at link time and passed via `Module.arguments`. Always serve over HTTP; `file://` fails to fetch the `.wasm`.

---

16. Troubleshooting

Nothing appears / controls invisible. You called the control outside `BeginDrawing()/EndDrawing()`, or after `EndDrawing()`. Every control call belongs between the two, once per frame.

My state resets every frame. The state variable is declared *inside* the frame loop. Immediate mode means the wrapper keeps nothing — move the variable above `while (!WindowShouldClose())`.

The text box won't accept typing. You used the short `textBox:…maxLength:` variant, which pins `editMode` to `NO`. Use the `editMode:` variant and toggle your own flag when it returns `YES` (§10).

The button fires more than once per click / never fires. A button returns `YES` once per click (on mouse release over the button). If it fires repeatedly, you are probably calling the control twice per frame, or re-creating it under a changing condition. If it never fires, the click is landing on a control drawn *after* it (later calls paint and hit-test on top).

The style file didn't load. `loadStyleAtPath:` returned `NO` — the path is wrong (paths are relative to the process working directory, so run from the repo root). A *text* `.rgs` also needs its font/charset files next to it; use a binary `.rgs` to avoid that. raygui only logs on failure, which is why the wrapper pre-checks.

Duplicate-symbol linker errors (`GuiButton`, …). `RAYGUI_IMPLEMENTATION` is defined in more than one translation unit. It must be defined in exactly one file — the project already has it in `src/RayguiImplementation.m`. Never define it in your own code, and don't compile `raygui.h` with the macro anywhere else.

`libraylib.a` fails to parse at link time (`!<arch>` errors). `-x objective-c` leaked onto the link command, so clang treated the archive as source. Compile `.m` → `.o` and link in separate steps with no `-x` on the link line (the Makefile does exactly this).

Linux: `gnustep-config: command not found`. Install `gnustep-devel` (Debian/Ubuntu) or the equivalent GNUstep make/dev package for your distribution.

Linux: window doesn't open over SSH / in a container. No display. Run on the desktop session, use X forwarding, or rely on the `IsWindowReady()` guard to exit cleanly.

ESC closes my window unexpectedly. raylib maps ESC to "exit" by default. `SetExitKey(0)` disables it; pick your own key with `SetExitKey(KEY_Q)` etc.

Text looks wrong / boxes too small for the font. The default style assumes the default font. If you `setFont:` or load a style, text metrics change — use `[RGGui textWidth:]` to measure and size your rectangles accordingly.

The slider's label is cut off / not where I expected. The label is drawn to the *left* of the slider bar, outside its bounds — place the slider far enough right to leave room, or pass `nil` and draw your own label (§9.1).

I want a resizable window / the GUI looks blurry on a Retina display. Resizing is opt-in: call `SetConfigFlags(FLAG_WINDOW_RESIZABLE)` *before* `InitWindow()` (raylib call). All coordinates are window points; raylib scales the framebuffer for high-DPI displays automatically, so no coordinate changes are needed.

Two text boxes are in edit mode at once and the cursor jumps between them. raygui keeps one shared global cursor state for text editing. Only one text-entry control (`textBox`, `valueBox`, `spinner`, an open `dropdownBox`, or a text-input dialog) may be in edit mode at a time — when one enters edit mode, set the others' `editMode` flags to `NO`.

Passing `NULL` for an in/out pointer. In/out pointers (`active:`, `checked:`, `value:`, `selected:`, `color:`, …) must be non-NULL — the wrapper raises `NSParameterAssert` (an `NSInvalidArgumentException` in debug builds) instead of crashing inside raygui. The parameters that legitimately accept `nil` are marked `nullable` in `src/RayguiObjC.h` (`text:`/`title:`/`items:`/`buttons:`, `scrollPanel:`'s `scroll:`, the dialogs' `secretView:`/`pressedButton:`).

17. Further reading

- `README.md` — project overview, layout, build summary.
- `examples/demo/main.m` — the acceptance demo; the smallest complete frame loop with several controls.
- `src/RayguiObjC.h` — the full API with per-method comments (the reference this manual summarizes).
- `vendor/raygui/examples/` — raygui's own C examples (`controls_test_suite`, `scroll_panel`, …) for side-by-side behavior comparison.
- `vendor/raygui/styles/` — the bundled themes, each with a screenshot.
- `VERSIONS.md` — pinned upstream tags/SHAs and the cherry-picked fix.
- `TODO.md` — project progress log.
- Upstream docs: https://github.com/raysan5/raygui (raygui) and https://github.com/raysan5/raylib (raylib) — raylib's cheatsheet is the reference for the pass-through API (`InitWindow`, `BeginDrawing`, `DrawText`, `LoadFont`, `BeginScissorMode`, …).